00 01 02 03 04 05 06 07 08 09 0A **0B** 0C 0D

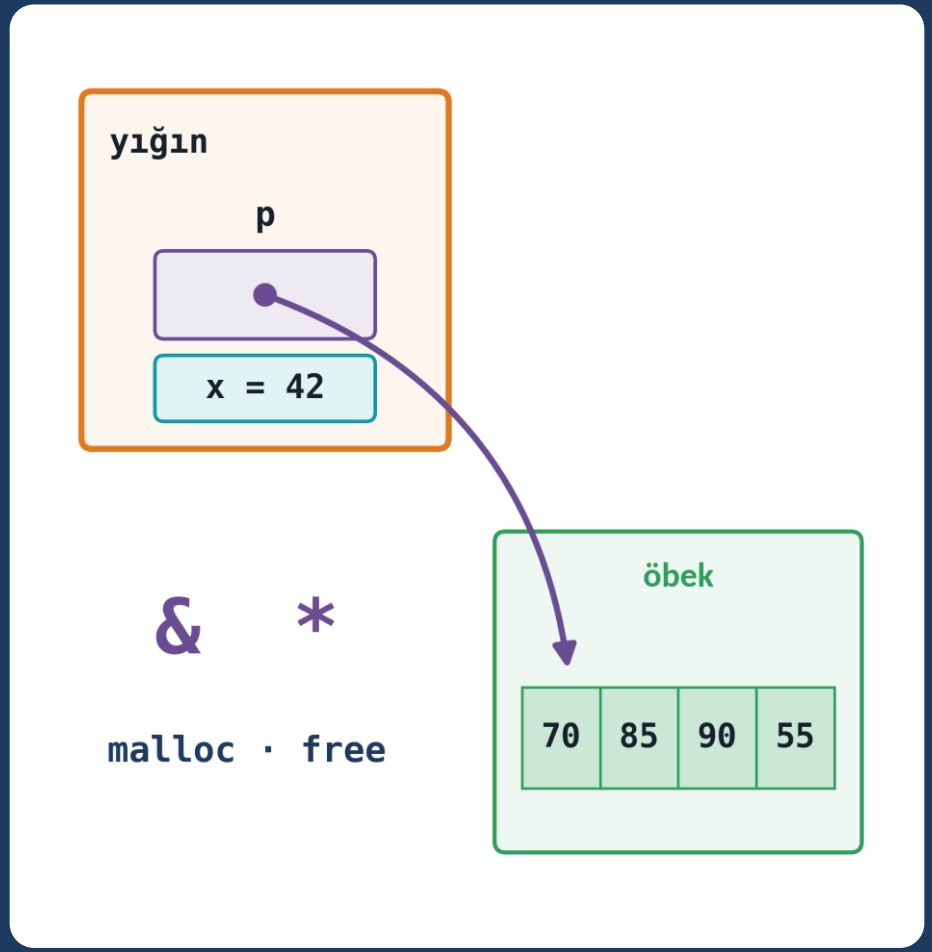
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 12

İşaretçiler: Adreslerle Programlama ve Belleğin Denetimi

Adres ve dolaylama · referansla aktarım · işaretçi aritmetiği ve diziler · yığın ve öbek · malloc/realloc/free · bellek hataları, AddressSanitizer ve Valgrind

MODÜL V · VERİ TOPLULUKLARI VE BELLEK



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** **İşaretçiler ve Bellek**	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H11 · SATIR ÖNCELİKLİ

`int m[3][4]`'te `m[2][1]`, `m[0][0]`'dan kaç bayt ileridedir? (`int` 4 bayt)

quiz/H12_Q01_satir_oncelikli.c · satır 12–20

```c
int main(void)
{
    int m[3][4] = {{0}};
    uintptr_t taban = (uintptr_t)&m[0][0];   /* adres → tamsayı */
    uintptr_t adres = (uintptr_t)&m[2][1];
    printf("m[2][1], m[0][0]'dan %lu bayt ileride\n",
           (unsigned long)(adres - taban));
    return 0;
}
```

A 36

B 24

C 9

D 44

GERİ ÇAĞIRMA · H11

S2 (2B dizi parametresi), S3 (komşuluk) ve gerçek yanıtlar

S2. `m = {{1, 2, 3}, {4, 5, 6}}`: `satir_toplami(m, 0)` ve `(m, 1)`? SUTUN neden zorunlu?

quiz/H12_Q02_2b_parametre.c · satır 10–17

```
10  int satir_toplami(int m[][SUTUN], int i)
11  {
12      int t = 0;
13      for (int j = 0; j < SUTUN; j++) {
14          t += m[i][j];
15      }
16      return t;
17  }
```

S3. Bütün hücreleri 1 olan 3 × 3 ızgarada köşe, kenar ve orta hücrenin 8-komşu toplamı?

Koşul: `(di != 0 || dj != 0) && a >= 0 && a < N && b >= 0 && b < N`

S1 · **A**: (2 × 4 + 1) × 4 = 36 bayt

S2: derleyici `m[i][j]` adresini taban + (i × SUTUN + j) × 4 ile bulur

S3 · **C**: sınır denetimi ızgara dışını atlar

GERÇEK ÇIKTILAR

```
$ ./q01
m[2][1], m[0][0]'dan 36 bayt ileride
$ ./q02
6 15
$ ./q03
köşe 3, kenar 5, orta 8
```

BÜYÜK SORU

Haftanın Büyük Sorusu

H8'de yazdığımız takas fonksiyonu neden çalışmadı ve bir değişkenin adresini bilmek bize hangi gücü ve hangi sorumluluğu verir?



Neden çalışmadı?

H8'in takası kopyalarla çalıştı. Fonksiyon main'in kutusuna nasıl uzanır?



Güç: adres

Adresi bilen kod başka çerçevedeki kutuya yazabilir, çalışma zamanında bellek isteyebilir.



Sorumluluk

Yanlış ya da süresi dolmuş adres: çökme ya da sessizce bozulan veri.

*Haftanın sonunda yanıtı **gerçek adreslerle** ve **araç raporlarıyla** kanıtlayacağız.*

GERÇEK DÜNYADAN

1965 → 2009

Hoare’un “milyar dolarlık hatası”

- 1965: **Tony Hoare**, ALGOL W’nin başvuru tiplerine **null referansı** ekler
- 2009, QCon London: “my billion-dollar mistake”; neden: “simply because it was so easy to implement”
- C’deki karşılığı: **NULL işaretçi**; dolaylamak **tanımsız davranış** (Linux/Windows’ta genellikle SIGSEGV, HA4)
- Bugün: bellek güvenliği hataları hâlâ güvenlik açıklarının en büyük grubu

≈ %70

Chromium: 912 yüksek/kritik güvenlik hatasında bellek güvenliği payı (2015’ten beri)

½

bunların yarısı free sonrası kullanım (use-after-free)

≈ %70

Microsoft: her yıl CVE atanan açıklarda bellek güvenliği payı (2019)

Hoare (2009), QCon London · Chromium Project (t.y.), Memory safety · MSRC Team (2019)

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

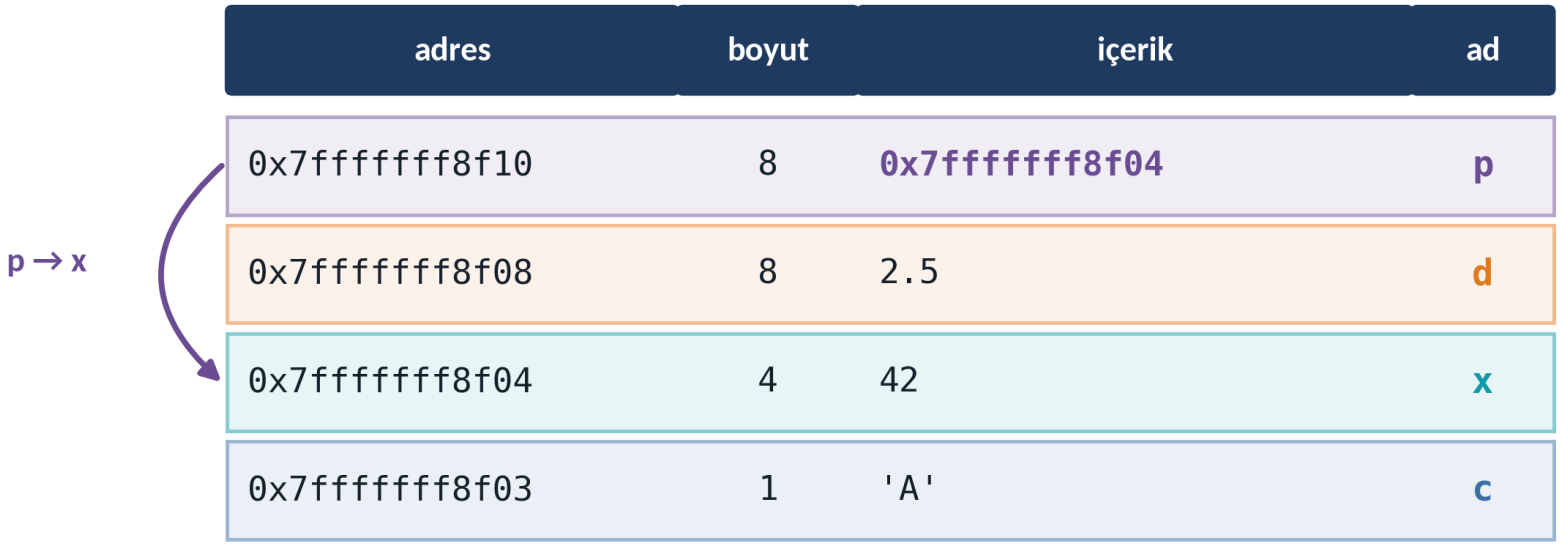
ÖÇ 12.1	İşaretçi bildirir; adres (&) ve dolaylama (*) operatörlerini doğru kullanır ve işaretçi–değer ilişkisini kutu–ok diyagramıyla gösterir.
ÖÇ 12.2	Referansla aktarım (işaretçi parametreler) ile fonksiyondan birden çok sonuç döndürür.
ÖÇ 12.3	İşaretçi aritmetiğini ve dizi–işaretçi ilişkisini (a[i] ≡ *(a + i)) açıklar.
ÖÇ 12.4	NULL, başlatılmamış ve sarkan (dangling) işaretçi hatalarını tanır ve önler.
ÖÇ 12.5	Yığın (stack) ve öbek (heap) belleğini ayırt eder; malloc/calloc/realloc/free ile dinamik dizi oluşturur ve yönetir.

01

Adres ve işaretçi

Bellek numaralı baytlardır · & ve * · kutu–ok modeli · NULL

Bellek: numaralı baytlardan oluşan bir dizi



- Her bayt bir **numara** (adres) taşır
- Çok baytlı değişkenin adresi **ilk baytıdır**
- %p adresi onaltılık yazar
- Şekil: **ASLR kapalı** (`setarch -R`) kararlı adresler
- Farklar değişmez: p, x'ten 12 bayt yukarıda

Yüksek adres üstte (H12_K02, setarch -R). p'nin İÇERİĞİ = x'in ADRESİ: kutu–ok modelindeki ok.

ASLR AÇIK: ADRESLER HER ÇALIŞTIRMADA DEĞİŞİR

```
$ ./adres_tablosu      # 1. çalıştırma
x 0x7fff1fff87e4  …
$ ./adres_tablosu      # 2. çalıştırma
x 0x7ffff072a254  …
```

& adres verir, * gösterilen nesneye gider: Örnek 1

H12_01_adres_deger.c · satır 15–22

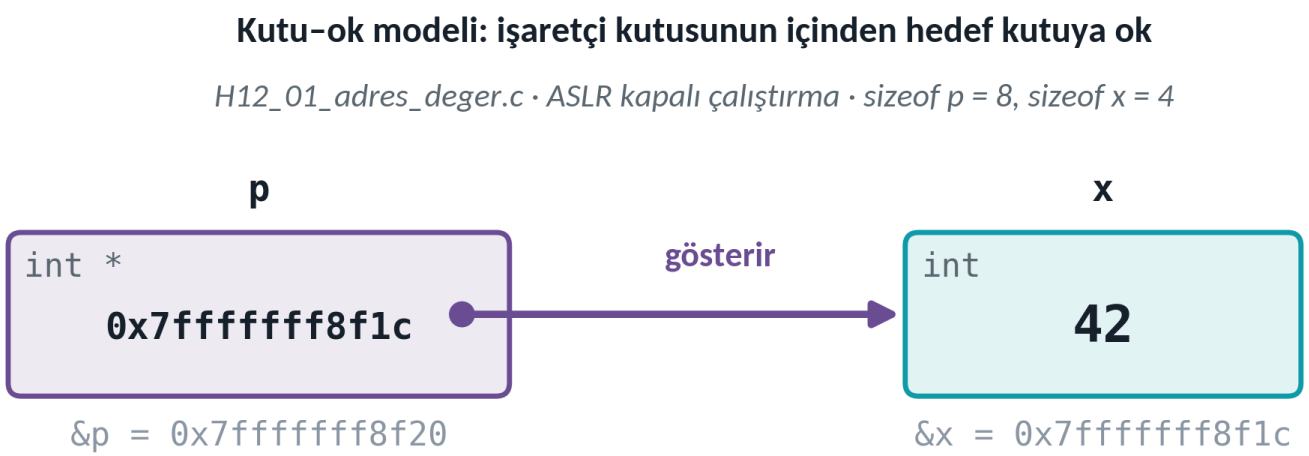
```c
15      int x = 42;
16      int *p = &x;                    /* p, x'in adresini tutar */
17
18      printf("x  (değer)              : %d\n", x);
19      printf("&x (x'in adresi)        : %p\n", (void *)&x);
20      printf("p  (içindeki adres)     : %p\n", (void *)p);
21      printf("*p (gösterdiği değer)   : %d\n", *p);
22      printf("&p (p'nin kendi adresi) : %p\n", (void *)&p);
```

GERÇEK ÇIKTI

```
$ setarch x86_64 -R ./adres_deger   # ASLR kapalı
x  (değer)              : 42
&x (x'in adresi)        : 0x7fffffff8f1c
p  (içindeki adres)     : 0x7fffffff8f1c
*p (gösterdiği değer)   : 42
&p (p'nin kendi adresi) : 0x7fffffff8f20
sizeof x = 4 bayt, sizeof p = 8 bayt
```

- `int *p` : “p, bir int’i gösterir”
- `p` ile `&x` **aynı** sayı; `*p` ile `x` **aynı** kutu
- İşaretçi 8 bayt: `sizeof(char *)` = `sizeof(double *)` = 8 (K01)

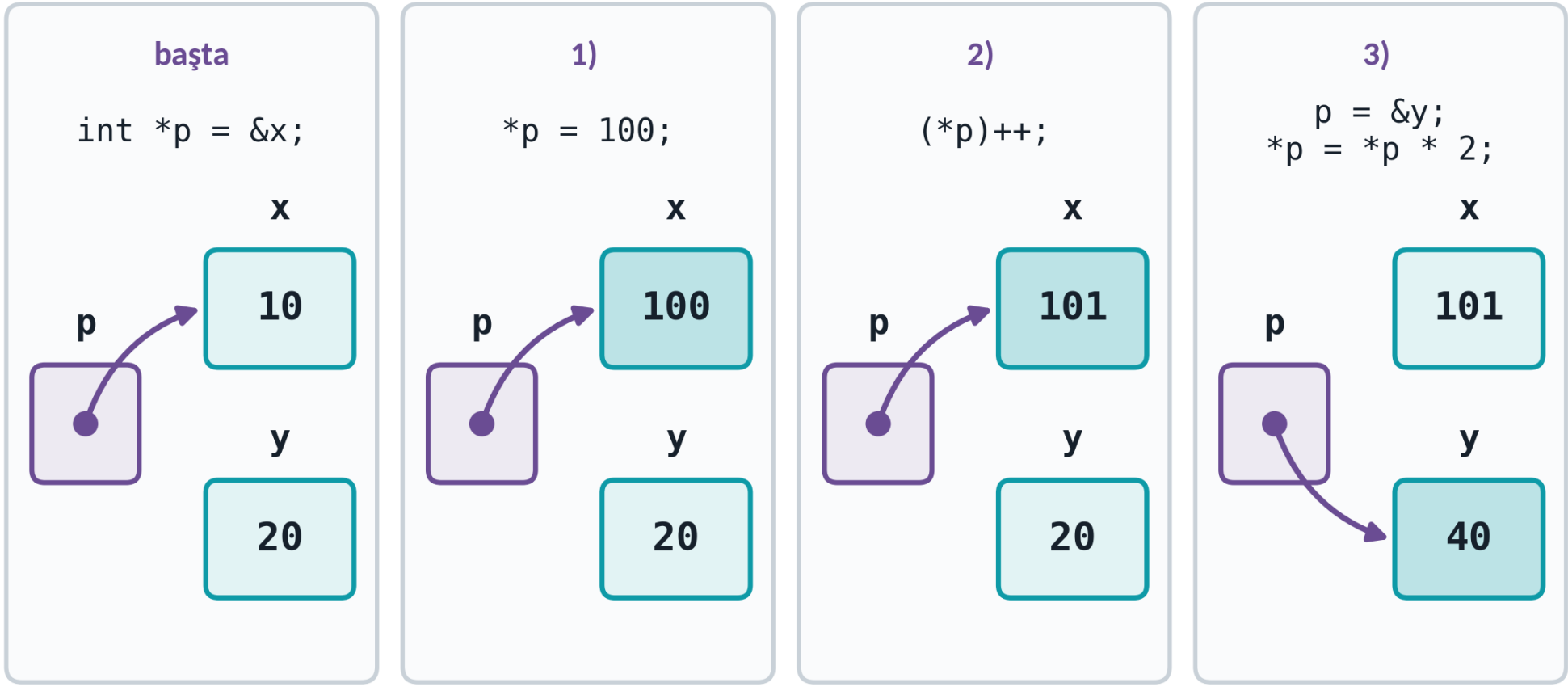
Kutu–ok modeli: işaretçi bir kutu, içinden bir ok çıkar




```
int *p = &x;   →  p'nin içine x'in ADRESİ yazılır (8 bayt)
*p             →  okun ucundaki kutu: x'in kendisi (42)
```

- İşaretçi **kutusunun içinde** bir adres durur
- Ok, o adresteki **kutuya** gider
- *p = okun ucundaki kutu
- Her satırdan sonra diyagramı **yeniden çizin**
- Python Tutor'un C görselleştiricisi aynı modeli kullanır

Dolaylama ile yazma: Örnek 2 adım adım



*p = … okun ucundaki kutuya yazar · p = &y okun yönünü değiştirir · (*p)++ ≠ *p++

TAHMİN ET · DOLAYLAMA İLE YAZMA

`*p += 3;` ardından `int *q = p; *q = *q * 2;` Program ne yazdırır?

quiz/H12_Q04_dolaylama.c · satır 6–15

```
 6  int main(void)
 7  {
 8      int x = 5;
 9      int *p = &x;
10      *p += 3;
11      int *q = p;
12      *q = *q * 2;
13      printf("x = %d\n", x);
14      return 0;
15  }
```

A `x = 5`

B `x = 8`

C `x = 16`

D `x = 10`

NULL: “hiçbir şeyi göstermiyor” ve güvenli ilk değer

H12_K07_null_guvenli.c · satır 15–23

```c
int *bul(int *a, int n, int aranan)
{
    for (int i = 0; i < n; i++) {
        if (a[i] == aranan) {
            return &a[i];
        }
    }
    return NULL;
}
```

H12_K07_null_guvenli.c · satır 30–33

```c
    p = bul(a, 3, 8);
    if (p != NULL) {                    /* dolaylamadan ÖNCE denetle */
        *p = 80;
    }
```

GERÇEK ÇIKTI

```
$ ./null_guvenli
a[1] = 80
99 yok (NULL döndü)
free(NULL) sorunsuz.
```

- İlk değer: adres **ya da** `NULL`
- Dolaylamadan **önce** denetle
- `free` sonrası `p = NULL`
- İlk değersiz işaretçi: **çöp adres** (HA3)

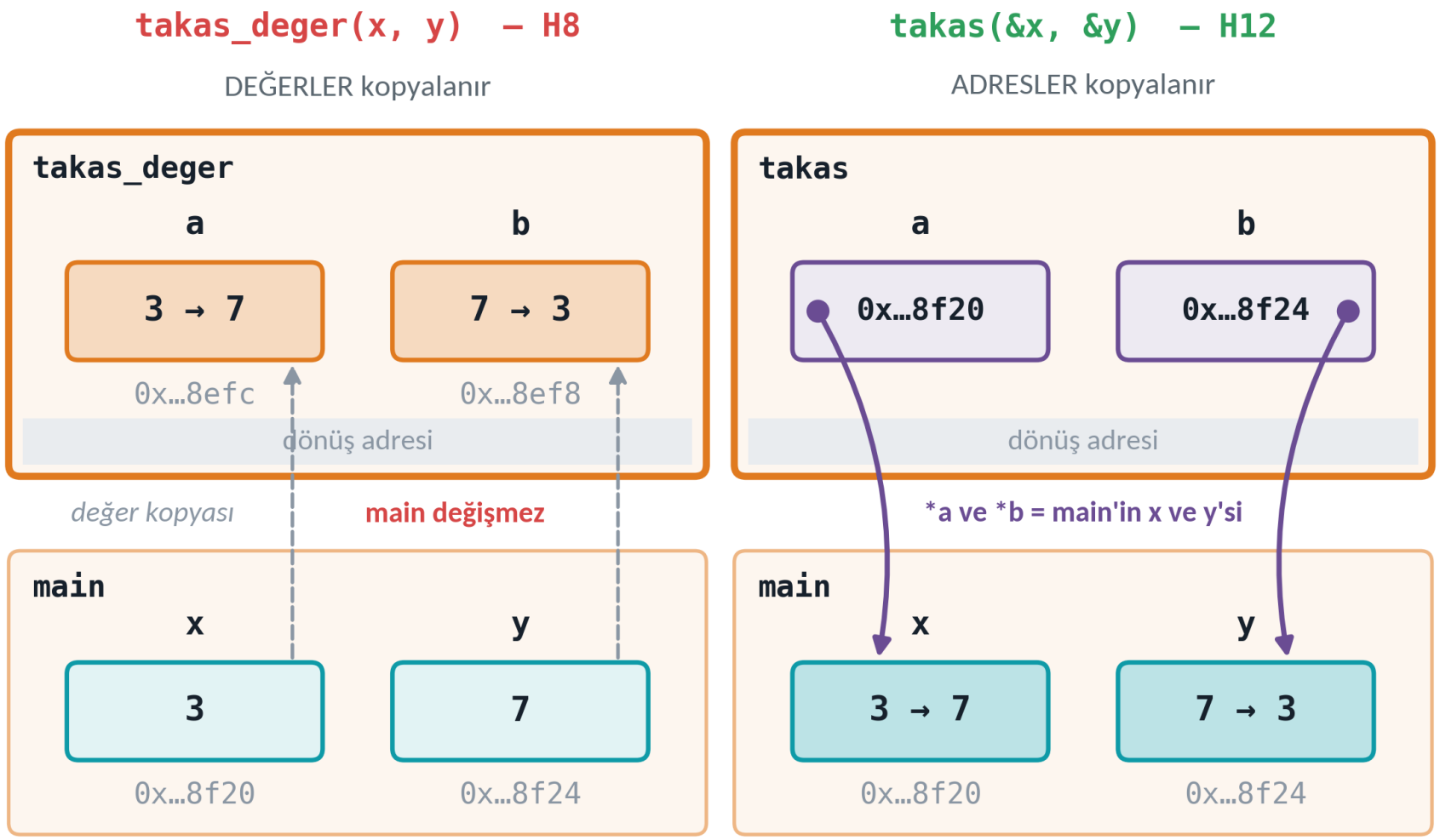
Aynı dosya: `free(blok); blok = NULL; free(blok);` → ikinci `free` hiçbir şey yapmaz (ISO C 7.22.3.3).

02

Referansla aktarım

takas çalışıyor · scanf’teki & · birden çok sonuç döndürmek

Değerle ve referansla takas: yığın çerçeveleri ve gerçek adresler



Örnek 3: takas(&x, &y) gerçekten değiştirir

H12_03_takas.c · satır 22–28

```
22  void takas(int *a, int *b)
23  {
24      printf("  takas      :  a = %p,  b = %p\n", (void *)a, (void *)b);
25      int gecici = *a;
26      *a = *b;
27      *b = gecici;
28  }
```

H12_03_takas.c · satır 32–37

```
32      int x = 3, y = 7;
33      printf("  main       : &x = %p, &y = %p\n", (void *)&x, (void *)&y);
34      takas_deger(x, y);
35      printf("takas_deger'den : x = %d, y = %d   <- değişmedi\n", x, y);
36      takas(&x, &y);
37      printf("takas'tan sonra : x = %d, y = %d   <- değişti\n", x, y);
```

- H3'ün takası: x → *a, y → *b
- Çağıran **adres** verir
- `takas(x, y)`: GCC 13 **uyarı**, GCC 14 **hata**
- `takas(&t, &t)` de doğru

Örnek 3: izleme tablosu, gerçek çıktı ve test

GERÇEK ÇIKTI (ASLR KAPALI)

```
$ setarch x86_64 -R ./takas
  main       : &x = 0x7fffffff8f20, &y = 0x7fffffff8f24
  takas_deger: &a = 0x7fffffff8efc, &b = 0x7fffffff8ef8
takas_deger'den : x = 3, y = 7   <- değişmedi
  takas      :  a = 0x7fffffff8f20,  b = 0x7fffffff8f24
takas'tan sonra : x = 7, y = 3   <- değişti
```

TEST: NORMAL, EŞİT, NEGATİF, TAKAS(&T, &T)

```
$ ./takas_testleri
takas: 4 test geçti.
```

Adım	Satır	Deyim	*a (x)	*b (y)	gecici
0	36	çağrı	3	7	—
1	25	gecici = *a	3	7	3
2	26	*a = *b	7	7	3
3	27	*b = gecici	7	3	3

- `takas`'ın `a`'sı = main'in `&x`: **aynı adres**
- `takas_deger`'in `&a`'sı farklı: **kopya**
- Her adımda **tek** kutu değişir

TAHMİN ET · İŞARETÇİ DE DEĞERLE GİDER

`yonlendir(&x, &y)` içinde `p = yeni; *p = 99;` Sonra x ve y?

quiz/H12_Q06_isaretci_kopyasi.c · satır 7–11

```c
7  void yonlendir(int *p, int *yeni)
8  {
9      p = yeni;                    /* yalnızca yerel kopya değişir */
10     *p = 99;
11 }
```

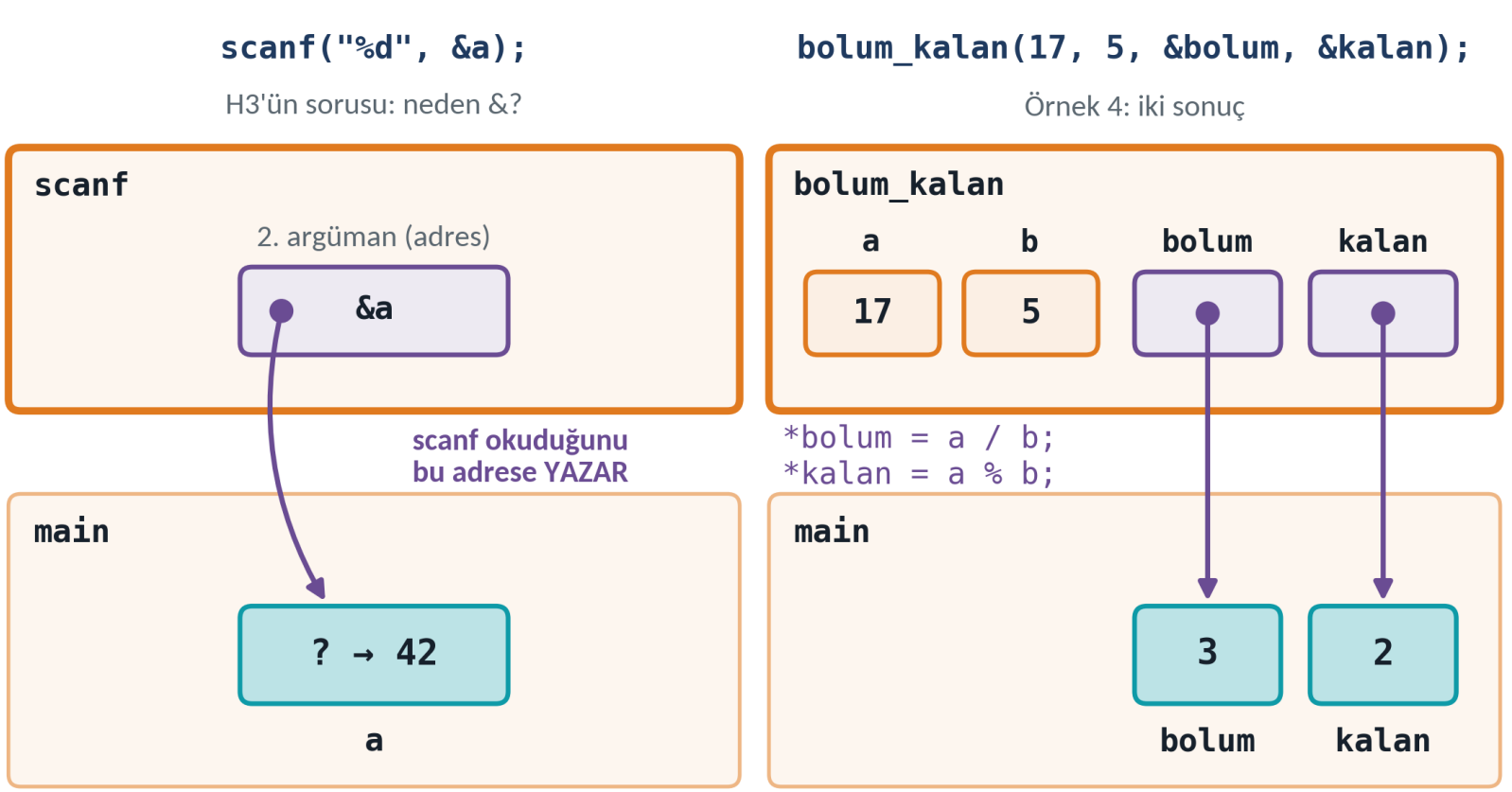
A `x = 99, y = 2`

B `x = 1, y = 2`

C `x = 99, y = 99`

D `x = 1, y = 99`

scanf'teki & ve çıktı parametreleri




```
H12_K04 · SCANF("%D", P)
$ ./scanf_adres
Bir sayı      : 5
Bir sayı daha : 9
x = 9
```

- scanf okuduğunu bir yere **yazmalı**: adres ister
- p zaten adres: & yazılmaz
- return tek değer; ötekiler **çıktı parametreleri**
- Dönüş değeri: **başarı** bilgisi

Örnek 4: bolum_kalan — iki sonuç, bir başarı bilgisi

H12_04_bolum_kalan.c · satır 18–26

```c
18  bool bolum_kalan(int a, int b, int *bolum, int *kalan)
19  {
20      if (b == 0) {
21          return false;
22      }
23      *bolum = a / b;
24      *kalan = a % b;
25      return true;
26  }
```

H12_04_bolum_kalan.c · satır 36–41

```c
36      int bolum, kalan;
37      if (bolum_kalan(a, b, &bolum, &kalan)) {
38          printf("%d = %d * %d + %d\n", a, b, bolum, kalan);
39      } else {
40          printf("Sıfıra bölme: bölüm ve kalan tanımsız.\n");
41      }
```

GERÇEK ÇIKTI

```
$ ./bolum_kalan
Bölünen ve bölen: 17 5
17 = 5 * 3 + 2

$ ./bolum_kalan
Bölünen ve bölen: -17 5
-17 = 5 * -3 + -2

$ ./bolum_kalan
Bölünen ve bölen: 7 0
Sıfıra bölme: bölüm ve kalan
    tanımsız.
```

- b = 0: çıktılara **dokunmaz**, `false`
- −17 / 5 = −3, −17 % 5 = −2 (sıfıra doğru)

Örnek 5: dizi_min_max — tek geçişte iki sonuç

H12_05_min_max.c · satır 13–25

```
13  void dizi_min_max(const int dizi[], int n, int *min, int *max)
14  {
15      *min = dizi[0];
16      *max = dizi[0];
17      for (int i = 1; i < n; i++) {
18          if (dizi[i] < *min) {
19              *min = dizi[i];
20          }
21          if (dizi[i] > *max) {
22              *max = dizi[i];
23          }
24      }
25  }
```

GERÇEK ÇIKTI

```
$ ./min_max
6 not: en küçük 38, en büyük
    90, aralık 52
1 not: en küçük 50, en büyük
    50
```

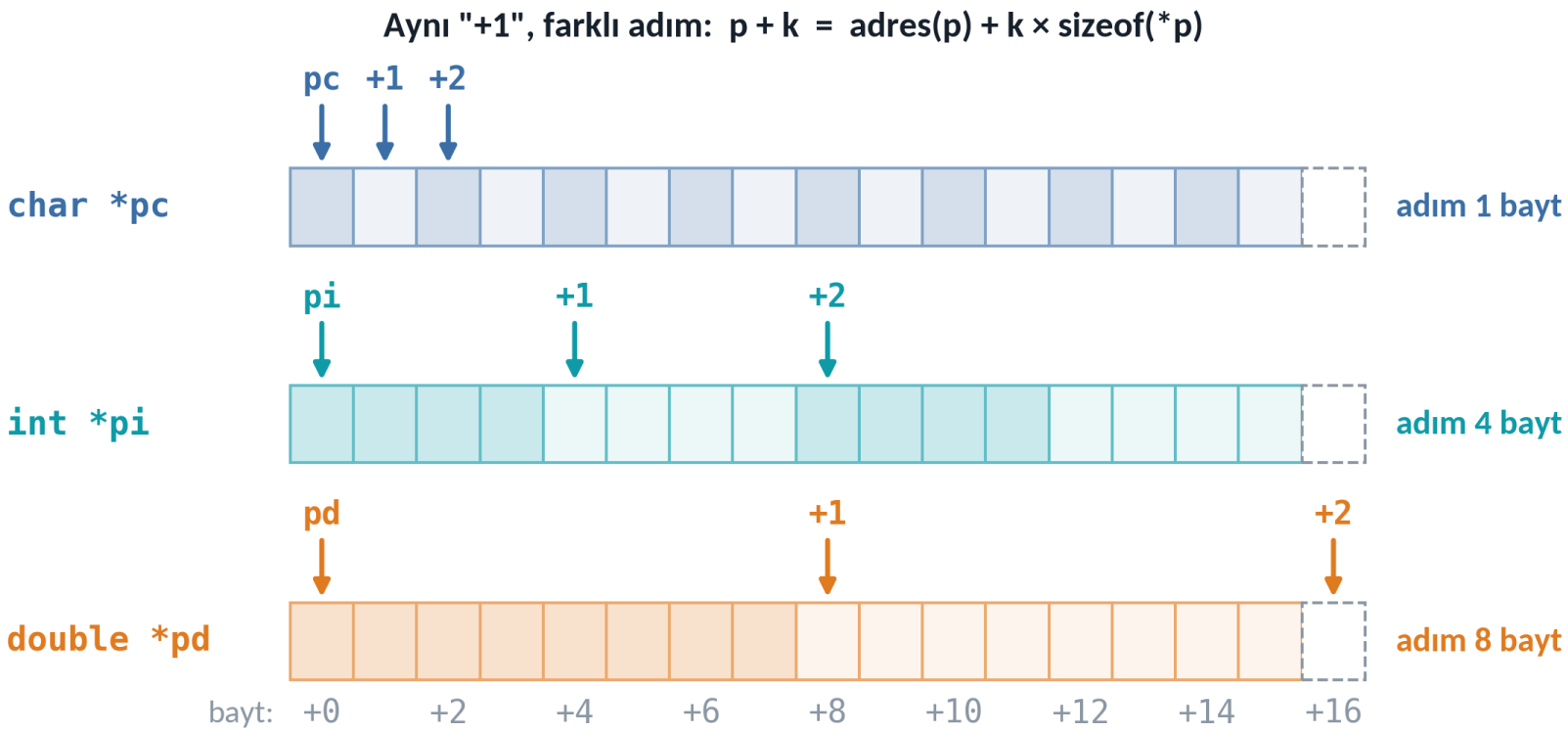
- Ara sonuçlar doğrudan çağıranın kutularında
- `const int dizi[]`: diziyi değiştirmeme sözü
- Ön koşul `n ≥ 1`; tek eleman sınır durumu
- Mini-OBS v0.12: `min_max_indis`

03

İşaretçi aritmetiği ve diziler

p + 1 kaç bayt? · a[i] ≡ *(a + i) · dizi parametresi · const

Aynı “+1”, farklı adım: ölçekleme



Gerçek ölçüm: H12_K03_olcekleme.c (+1 adımı 1 · 4 · 8 bayt); renk tonları elemanları ayırır

- `p + k` → adres + k × `sizeof(*p)`
- Adım **1 / 4 / 8** bayt: char / int / double
- `p - q`: aradaki **eleman** sayısı
- Yalnızca **aynı dizi** içinde anlamlı
- Adımlar K03’ün **gerçek ölçümü**

TAHMİN ET · İŞARETÇİ ARİTMETİĞİ

`p = a + 1` iken `*p + 1` ve `*(p + 1)` ne yazdırır?

quiz/H12_Q05_aritmetik.c · satır 6–12

```c
int main(void)
{
    int a[] = {10, 20, 30, 40};
    int *p = a + 1;
    printf("%d %d\n", *p + 1, *(p + 1));
    return 0;
}
```

A) `21 30`

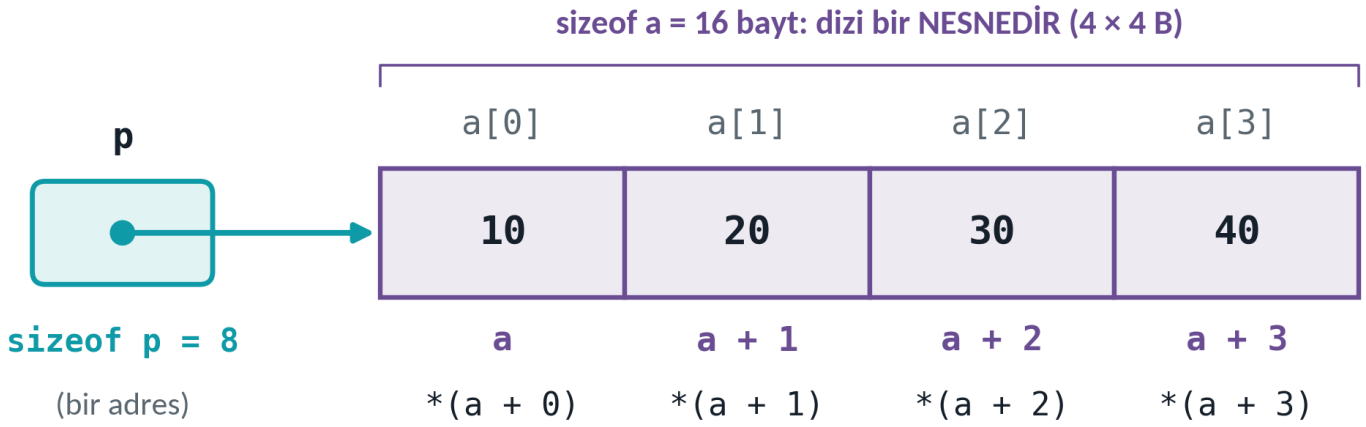
B) `20 30`

C) `11 21`

D) `21 31`

Dizi adı = ilk elemanın adresi: a[i] ≡ *(a + i)

```
int a[4] = {10, 20, 30, 40};   int *p = a;
```



a[i] ≡ *(a + i) ≡ p[i] ≡ *(p + i) ≡ i[a]

ifadede a → &a[0] dönüşür (a == &a[0] : evet) · istisnaları: sizeof a ve &a

- a → &a[0]
- **Tanım:** a[i] = *(a + i)
- p[i] ≡ *(p + i)
- sizeof a 16, sizeof p 8
- i[a] bile derlenir
- H11: m[i] = **satır adresi**

ÖRNEK 7 (H12_07) · SEÇİLMİŞ SATIRLAR

```
 i  a[i]  *(a + i)  p[i]  *(p + i)  i[a]
 0    10        10    10        10    10
 3    40        40    40        40    40
a == &a[0] ? evet
sizeof a = 16 bayt, sizeof p = 8 bayt
```

Örnek 6: p++ ile dizi dolaşma

H12_06_isaretci_aritmetigi.c · satır 15–23

```c
15      int a[5] = {10, 20, 30, 40, 50};
16      int *son = a + 5;                   /* son elemanın BİR ötesi */
17      int toplam = 0;
18
19      for (int *p = a; p < son; p++) {    /* p++ : bir sonraki int */
20          printf("p = %p   *p = %d   p - a = %td\n", (void *)p, *p, p - a);
21          toplam += *p;
22      }
23      printf("toplam = %d\n", toplam);
```

GERÇEK ÇIKTI

```
$ ./isaretci_aritmetigi
p = 0x7ffd87acdfb0   *p = 10   p - a = 0
p = 0x7ffd87acdfb4   *p = 20   p - a = 1
p = 0x7ffd87acdfb8   *p = 30   p - a = 2
p = 0x7ffd87acdfbc   *p = 40   p - a = 3
p = 0x7ffd87acdfc0   *p = 50   p - a = 4
toplam = 150
a + 1, a'dan 4 bayt ileride
```

- Döngü değişkeni bir **adres**: her p++ 4 bayt
- p - a: kaçıncı eleman
- a + 5 bir öte: **karşılaştırılır**, dolaylanmaz
- *p++ = *(p++): oku, sonra ilerle

Dizi parametresi aslında işaretçidir: H10'un sizeof tuzağı

H12_K05_dizi_parametresi.c · satır 14–23

```c
14  void boyut_yaz(int dizi[])              /* derleyici için: int *dizi */
15  {
16      printf("fonksiyonda sizeof dizi  = %zu  (int * boyutu)\n", sizeof dizi);
17  }
18
19  int main(void)
20  {
21      int notlar[10] = {0};
22      printf("main'de     sizeof notlar = %zu (10 x 4 bayt)\n", sizeof notlar);
23      boyut_yaz(notlar);                  /* giden: &notlar[0] */
```

GERÇEK DERLEYİCİ İLETİSİ VE ÇIKTI (DOSYA KASITLI UYARI İÇERİR)

```
satır 16:71: warning: ‘sizeof’ on array function parameter ‘dizi’ will return size of ‘int *’
    [-Wsizeof-array-argument]
$ ./dizi_parametresi
main'de     sizeof notlar = 40 (10 x 4 bayt)
fonksiyonda sizeof dizi  = 8  (int * boyutu)
```

Örnek 8: iki işaretçiyle ters çevirme

H12_08_ters_cevir_isaretci.c · satır 30–38

```
30  void ters_cevir(int *bas, int *son)
31  {
32      while (bas < son) {
33          printf("  takas: *bas = %d, *son = %d\n", *bas, *son);
34          takas(bas, son);
35          bas++;
36          son--;
37      }
38  }
```

GERÇEK ÇIKTI

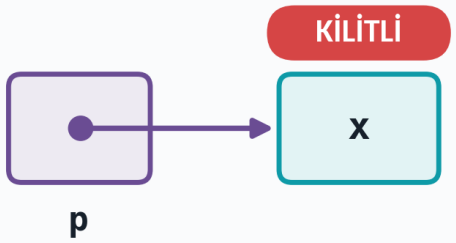
```
$ ./ters_cevir_isaretci
önce    : 1 2 3 4 5 6 7
  takas: *bas = 1, *son = 7
  takas: *bas = 2, *son = 6
  takas: *bas = 3, *son = 5
sonra   : 7 6 5 4 3 2 1
indisle : 1 2 3 4 5 6 7
  takas: *bas = 2, *son = 6
  takas: *bas = 3, *son = 5
ortası  : 1 6 5 4 3 2 7
```

- H10'un iki ucu (`i`, `j` indisleri) şimdi iki **adres**
- `takas(bas, son)`: adresler zaten elde, `&` yok
- Alt diziye uygulanır: `ters_cevir(a + 1, a + n - 2)`
- AP-2: "two pointers" yöntemi

const ve işaretçiler: neyin kilitlendiğini yer belirler

`const int *p`

gösterilen kilitli

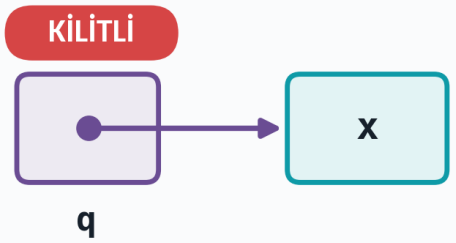


`p = &y;` ✓ geçerli

`*p = 5;` ✗ hata

`int *const q`

işaretçi kilitli

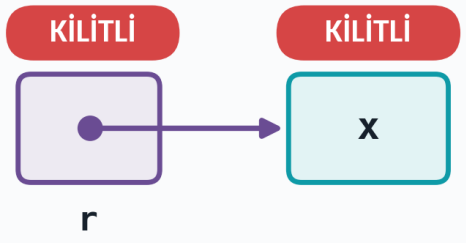


`*q = 10;` ✓ geçerli

`q = &y;` ✗ hata

`const int *const r`

ikisi de kilitli



`*r = 1;` ✗ hata

`r = &x;` ✗ hata

- Bildirimi **sağdan sola** okuyun
- `const int *p`: gösterilen kilitli
- `int *const q`: işaretçi kilitli
- Salt okuyan parametre: `const int *a` (**söz**)
- Örnek 9 · H12_09

GERÇEK DERLEYİCİ İLETİLERİ (H12_K06, KASITLI)

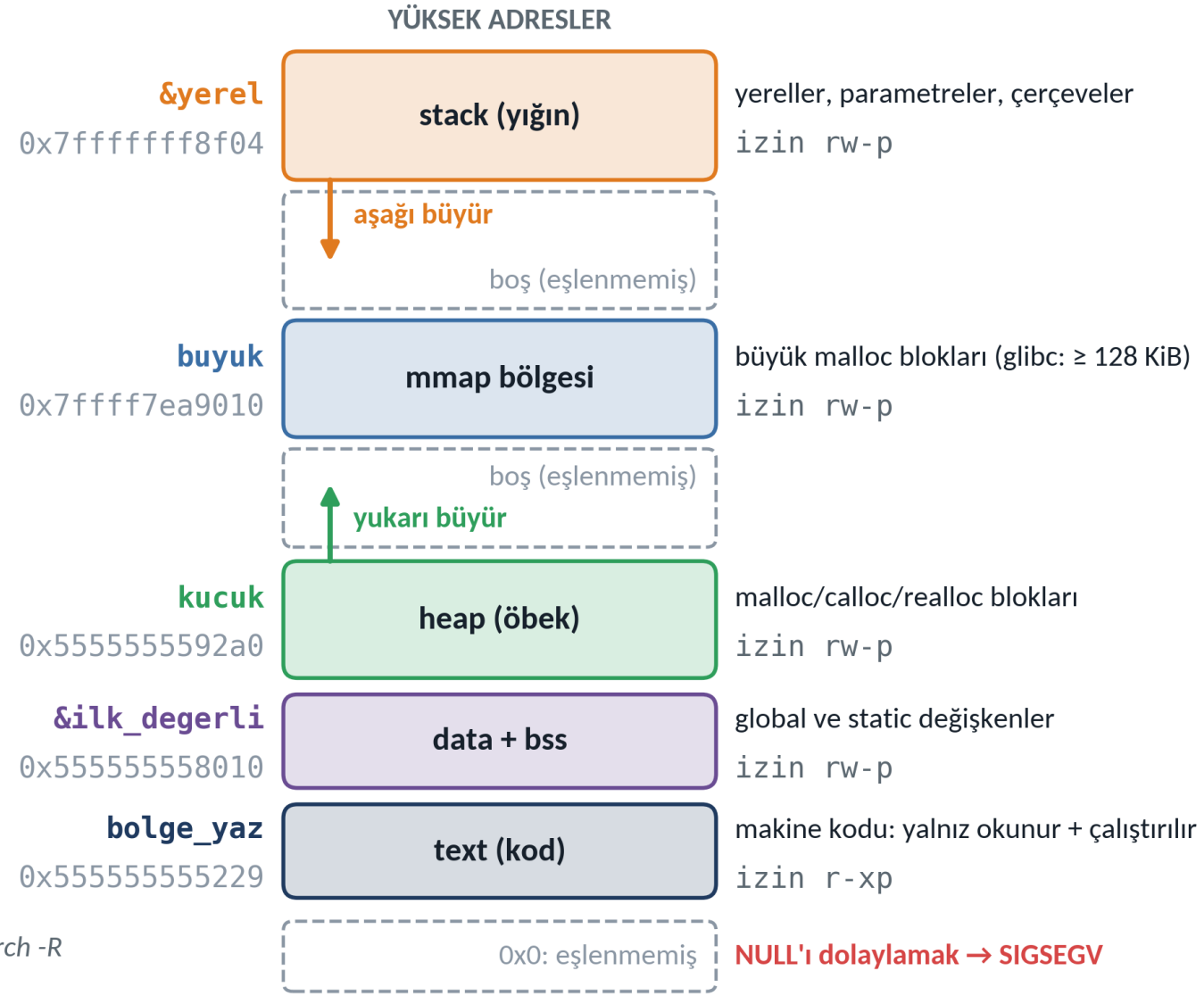
```
satır 25:8: error: assignment of read-only location ‘*p’
satır 27:7: error: assignment of read-only variable ‘q’
```

04

Yığın, öbek ve dinamik bellek

Süreç bellek haritası · sanal bellek · malloc, calloc, realloc, free

Kaputun altında: tamamlanan süreç bellek haritası



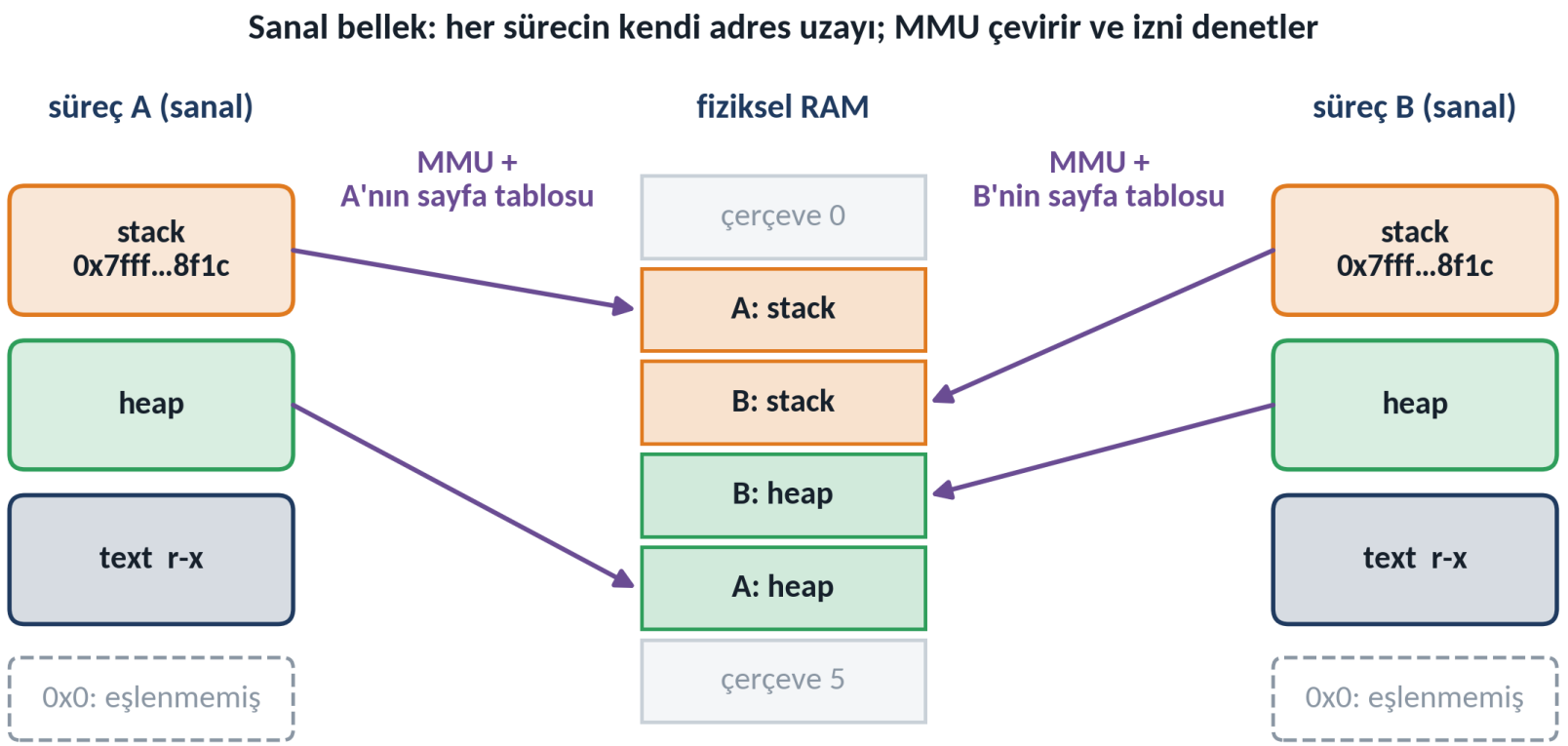
H12_K10 · setarch -R
gerçek adresler

- **Yığın**: yerel değişkenler, çerçeveler; ömür otomatik, boyut küçük (tipik 8 MiB)
- **Öbek**: çalışma zamanında istenen bloklar; ömrü programcı yönetir
- Büyük bloklar (≥ 128 KiB) ayrı **mmap** bölgesinden
- Bölgeler **4 KiB'lık sayfalar**

GERÇEK ADRES VE İZİN

```
text  0x555555555229 r-xp
heap  0x5555555592a0 rw-p
stack 0x7fffffff8f04 rw-p
```

Sanal bellek, MMU ve “Segmentation fault”



Aynı sanal adres (0x7fff...8f1c) iki süreçte FARKLI fiziksel çerçevelere eşlenir.

Eşlenmemiş sayfaya erişim ya da izinsiz erişim (ör. r-x sayfaya yazma) → işletim sistemi SIGSEGV gönderir

```
kabuk: Segmentation fault · çıkış durumu 128 + 11 = 139
```

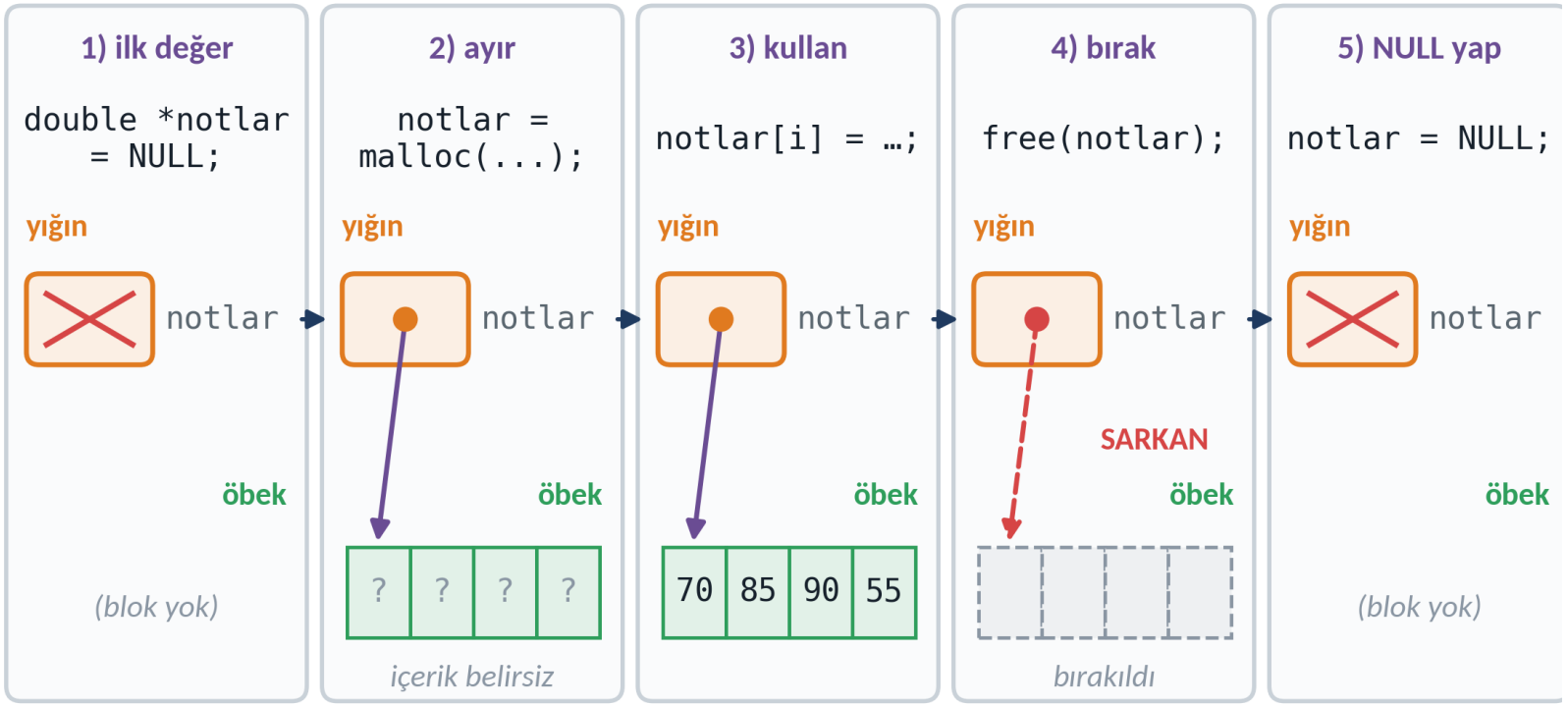
- Program **sanal** adres görür
- **MMU** her erişimi sayfa tablosuyla çevirir, izni denetler
- Eşlenmemiş ya da izinsiz erişim → **SIGSEGV**
- Çıkış durumu 128 + 11 = **139**

K11 · ÇIKIŞ DURUMLARI

```
$ ./koruma_ihlali
1) NULL'dan oku → 139
2) salt okunura yaz → 139
```

*Her hatalı erişim segfault **vermez**: dizinin bir ötesi çoğu zaman aynı izinli sayfadadır, **sessizce** bozar.*

Dinamik bellek: dört fonksiyon, dört kural



- `malloc(n)`: n bayt, içerik belirsiz
- `calloc(a, b)`: a × b bayt, **sıfırlı**
- `realloc(p, y)`: büyüt/küçült, gerekirse **taşı**
- `free(p)`: geri ver; `free(NULL)` etkisiz

- `void *` kendiliğinden dönüşür: C'de **cast gerekmez** · boyut `n * sizeof *p`
- Dönüşü **her zaman** NULL'a karşı denetle · her bloğun **tek sahibi**, **tek** `free`'si

BAŞARISIZLIK GERÇEKTİR

```
$ ./bellek_yetersiz   # K09
malloc(9223372036854775807)
    -> NULL
realloc başarısız; eski
    blok yerinde: 10 20 30
```

Örnek 10: malloc ile not dizisi

H12_10_malloc_not_dizisi.c · satır 23–37

```
23      double *notlar = malloc(n * sizeof *notlar);    /* öbekten n double */
24      if (notlar == NULL) {                           /* her zaman denetle */
25          printf("Bellek yetersiz.\n");
26          return 1;
27      }
28
29      double toplam = 0.0;
30      for (int i = 0; i < n; i++) {
31          printf("%d. not: ", i + 1);
32          if (scanf("%lf", &notlar[i]) != 1) {
33              printf("Geçersiz not.\n");
34              free(notlar);                           /* erken çıkışta da */
35              return 1;
36          }
37          toplam += notlar[i];
```

GERÇEK ÇIKTI

```
$ ./malloc_not_dizisi   # 4 not: 70 85 90 55 (seçilmiş satırlar)
Ortalama: 75.00
Ortalamanın üstü: 85.0 90.0
notlar (yığında) 0x7fff604a9870 -> blok (öbekte) 0x55985adcaac0
```

TAHMİN ET · SİZEOF D İLE SİZEOF *D

`sizeof d` ve `4 * sizeof *d` ne yazdırır?

quiz/H12_Q10_malloc_sizeof.c · satır 8–17

```c
int main(void)
{
    double *d = malloc(4 * sizeof *d);
    if (d == NULL) {
        return 1;
    }
    printf("%zu %zu\n", sizeof d, 4 * sizeof *d);
    free(d);
    return 0;
}
```

A 32 32

B 8 32

C 8 8

D 4 32

Örnek 12: realloc ile büyüyen dizi

H12_12_realloc_dinamik_dizi.c · satır 28–40

```c
        if (n == kapasite) {                    /* dolu: ikiye katla */
            uintptr_t eski = (uintptr_t)dizi;   /* adresi ÖNCE sakla */
            int *gecici = realloc(dizi, 2 * kapasite * sizeof *dizi);
            if (gecici == NULL) {               /* dizi hâlâ geçerli */
                printf("Bellek yetersiz: %d elemanda kaldı.\n", n);
                break;
            }
            dizi = gecici;
            kapasite *= 2;
            int tasindi = (uintptr_t)dizi != eski;
            tasinma += tasindi;
            printf("n = %2d: kapasite %2d -> %2d  %s\n", n, kapasite / 2,
                   kapasite, tasindi ? "(blok taşındı)" : "(yerinde büyüdü)");
```

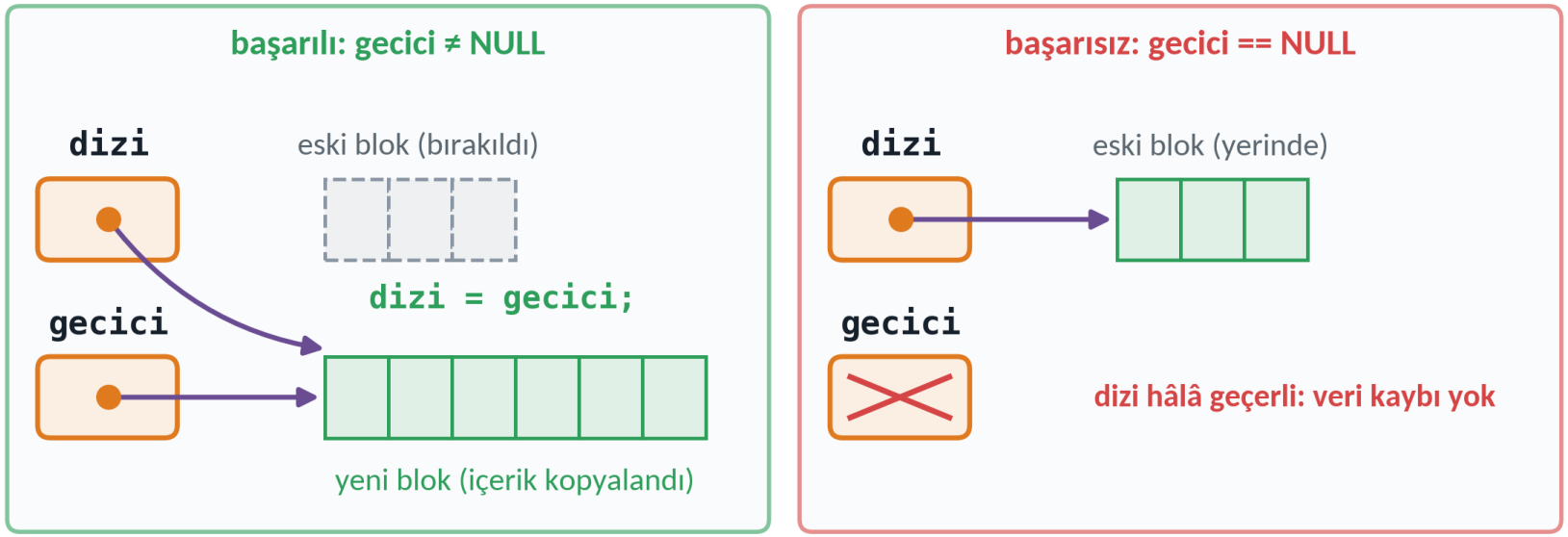
GERÇEK ÇIKTI

```
$ ./realloc_dinamik_dizi   # seçilmiş satırlar
n =  1: kapasite  1 ->  2  (yerinde büyüdü)
n =  4: kapasite  4 ->  8  (blok taşındı)
n = 16: kapasite 16 -> 32  (yerinde büyüdü)
20 eleman, kapasite 32, 1 taşınma; son eleman 400
```

- 5 büyütme; en çok 31 kopya
- Valgrind altında **5/5** taşındı

realloc'u güvenle kullanmak: geçici işaretçi

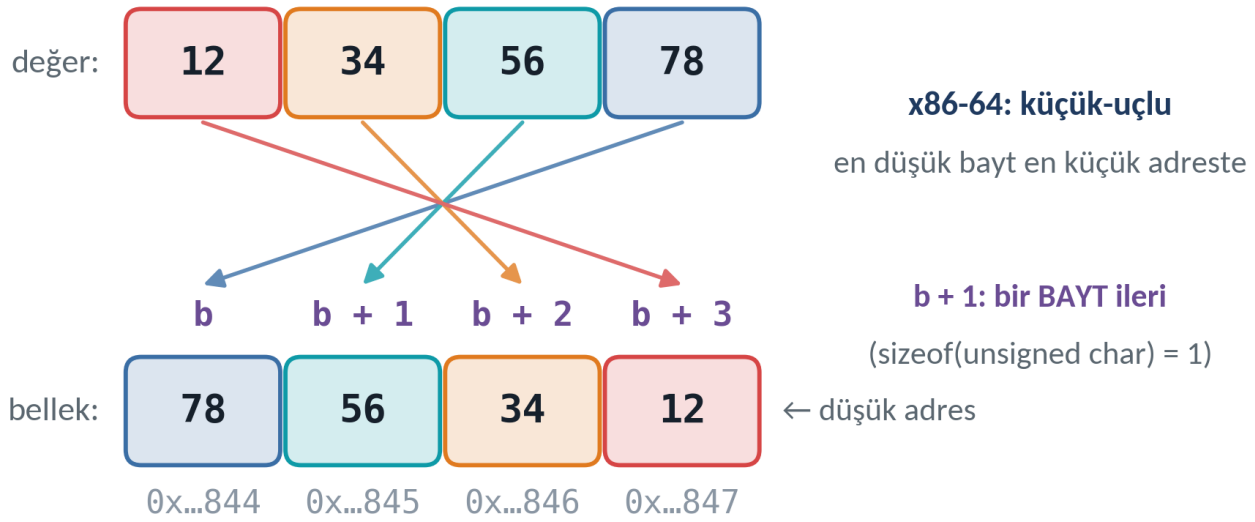
```
int *gecici = realloc(dizi, yeni_boyut);
```



- `dizi = realloc(dizi, …)`: NULL dönerse **tek adres kaybolur** → sızıntı + veri kaybı
- Doğrusu: `gecici` → denetle → `dizi = gecici`
- Başarılı realloc'tan sonra **eski işaretçi kullanılmaz**
- Mini-OBS v0.12: dört dizi, dört geçici işaretçi

Örnek 11: char * ile bir int'in baytları

```
int sayi = 0x12345678;   (anlamlılık: 12 en yüksek, 78 en düşük)
```




```
unsigned char *b = (unsigned char *)&sayi;
```

GERÇEK ÇIKTI

```
$ ./bayt_inceleme   # son iki satır
İlk bayt 0x78: bu makine küçük-uçlu.
b[3] = 0x7F sonrası sayi = 0x7F345678
```

- `unsigned char *b = (unsigned char *)&sayi;`
- x86-64 **küçük-uçlu**: bellekte 78 56 34 12
- `b + k` **bir bayt** ilerler
- Karakter tipiyle bakmak C'de izinli
- H3'te gdb ile görmüştük (`H03_K02`)

05

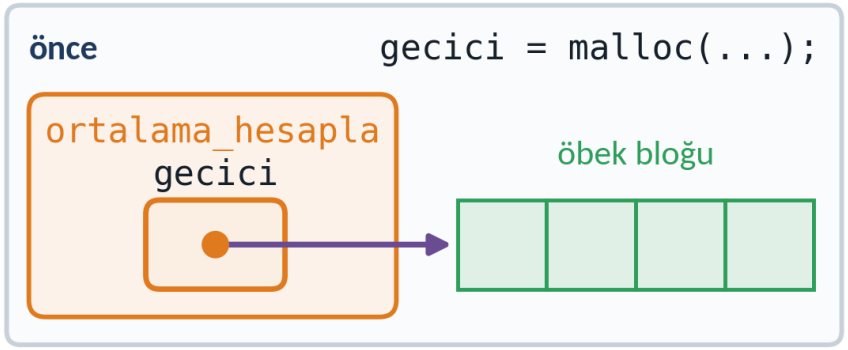
Bellek hataları ve araçlar

Sızıntı ve sarkan işaretçi · AddressSanitizer · Valgrind · Hata Avcısı

Sızıntı ve sarkan işaretçi: birbirinin aynadaki görüntüsü

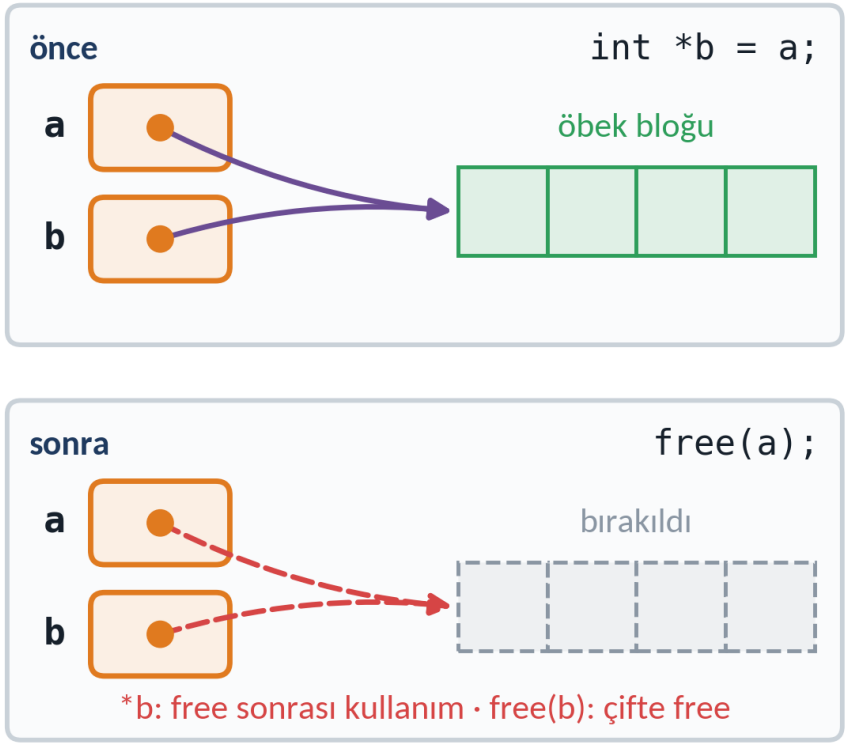
Bellek sızıntısı

blok var, onu gösteren işaretçi yok



Sarkan işaretçi

işaretçi var, gösterdiği blok yok



Sızıntı: blok var, işaretçi yok · Sarkan: işaretçi var, blok yok → free sonrası kullanım, çifte free

AddressSanitizer raporunu okumak

AddressSanitizer raporunu okumak (H12_HA5_free_sonrasi.c, kısaltılmış gerçek rapor)

```
ERROR: AddressSanitizer: heap-use-after-free

READ of size 8 at 0x503000000040 thread T0

  #0 ... in main H12_HA5_free_sonrasi.c:21

0x… is located 0 bytes inside of 24-byte region […]

freed by thread T0 here:

  #1 ... in main H12_HA5_free_sonrasi.c:20

previously allocated by thread T0 here:

  #1 ... in main H12_HA5_free_sonrasi.c:13

SUMMARY: AddressSanitizer: heap-use-after-free …
```

1. hata türü
2. erişim: okuma, 8 bayt
3. NEREDE erişildi
4. hangi blok
5. NEREDE bırakıldı
6. NEREDE ayrıldı
7. özet

Örnek 13: aynı beş hata, ASan'lı ve ASan'sız

Seçim	Hata	ASan'lı: rapor	ASan'sız: gözlenen	Çıkış
1	sınır dışı yazma (a[4])	heap-buffer-overflow	sessiz: "normal" bitti	0
2	free sonrası okuma	heap-use-after-free	*p = 1624208527	0
3	çifte free	attempting double-free	free(): double free detected in tcache 2 · Aborted	134
4	sızıntı (400 bayt)	detected memory leaks	sessiz: "normal" bitti	0
5	NULL'a yazma	SEGV	Segmentation fault	139

- ASan'sız: beş hatanın **ikisi hiç görünmedi** (çıkış 0), biri anlamsız değer yazdı
- ASan: **dördü ilk anda**, sızıntı **çıkışta** (LSan); derleyici ikisini `-Wuse-after-free` ile önceden uyardı

* ile & karıştırmak: uyarı yeter mi?

✗ KASITLI HATALI · H12_HA1_yildiz_ampersand.c · satır 14–16

```
14      int y = p;                  /* HATA: * unutuldu → y'ye ADRES gider */
15      printf("y = %d\n", y);
16      p = x;                      /* HATA: & unutuldu → p'ye 5 yazılır */
```

✓ DÜZELTİLMİŞ · H12_HA1_yildiz_ampersand_duzeltilmis.c · satır 13–15

```
13      int y = *p;                 /* DOĞRU: gösterilen DEĞER */
14      printf("y = %d\n", y);
15      p = &x;                     /* DOĞRU: x'in ADRESİ */
```

- Değer istenen yerde *, adres istenen yerde &
- GCC 13'te **uyarı**, GCC 14'te **hata**
- HA2: `int *p, q;` → q bir `int` (`sizeof q` = 4)

GERÇEK İLETİ VE ÇALIŞTIRMA

```
satır 14:13: warning: initialization of ‘int’ from ‘int *’ makes integer from pointer without a cast
satır 16:7: warning: assignment to ‘int *’ from ‘int’ makes pointer from integer without a cast
$ ./ha1
y = 1098683720
p = 0x5
Segmentation fault
çıkış durumu: 139
```

malloc(sizeof(kaynak)): işaretçinin boyutu kadar blok

✗ KASITLI HATALI · H12_HA7_malloc_sizeof.c · satır 13

```
13      int *yeni = malloc(sizeof(kaynak));     /* HATA 1: 8 bayt */
```

✓ DÜZELTİLMİŞ · H12_HA7_malloc_sizeof_duzeltilmis.c · satır 13–16

```
13      int *yeni = malloc(n * sizeof *yeni);   /* 5 × 4 = 20 bayt */
14      if (yeni == NULL) {
15          return NULL;
16      }
```

- `sizeof(kaynak)` = 8: **işaretçi**
- Beş `int` = 20 bayt yazılır
- İkinci hata: NULL denetimi yok
- ASan'sız: **doğru sonuç** (şans)

GERÇEK İLETİ VE ÇALIŞTIRMA

```
$ ./ha7
kopya[4] = 62
$ ./ha7_asan
AddressSanitizer: heap-buffer-overflow on address 0x502000000018
WRITE of size 4 at 0x502000000018 thread T0
0x502000000018 is located 0 bytes after 8-byte region [0x502000000010,0x502000000018)
```

Yerel değişkenin adresini döndürmek

✗ KASITLI HATALI · H12_HA8_yerel_adres.c · satır 12–16

```
12      int sonuc[10];                  /* yerel: çerçeveyle birlikte ölür */
13      for (int i = 0; i < n; i++) {
14          sonuc[i] = i * i;
15      }
16      return sonuc;                   /* HATA: yerel dizinin adresi */
```

✓ DÜZELTİLMİŞ · H12_HA8_yerel_adres_duzeltilmis.c · satır 14

```
14      int *sonuc = malloc(n * sizeof *sonuc);     /* öbekte: yaşamaya devam */
```

- Yerel dizi **çerçeveyle ölür**
- GCC yerel adres yerine **NULL** döndürdü
- Düzeltme: öbekte ayır, **çağıran** free etsin

GERÇEK İLETİ VE ÇALIŞTIRMA

```
satır 16:12: warning: function returns address of local variable [-Wreturn-local-addr]
$ ./ha8
k = (nil)
Segmentation fault
çıkış durumu: 139
```

Bellek sızıntısı: çıktı doğru, bellek kayıp

✗ KASITLI HATALI · H12_HA9_sizinti.c · satır 20–22

```
20          toplam += gecici[i];
21      }
22      return (double)toplam / n;      /* HATA: free(gecici) unutuldu */
```

✓ DÜZELTİLMİŞ · H12_HA9_sizinti_duzeltilmis.c · satır 23–24

```
23      free(gecici);                   /* DOĞRU: dönmeden önce bırak */
24      return ortalama;
```

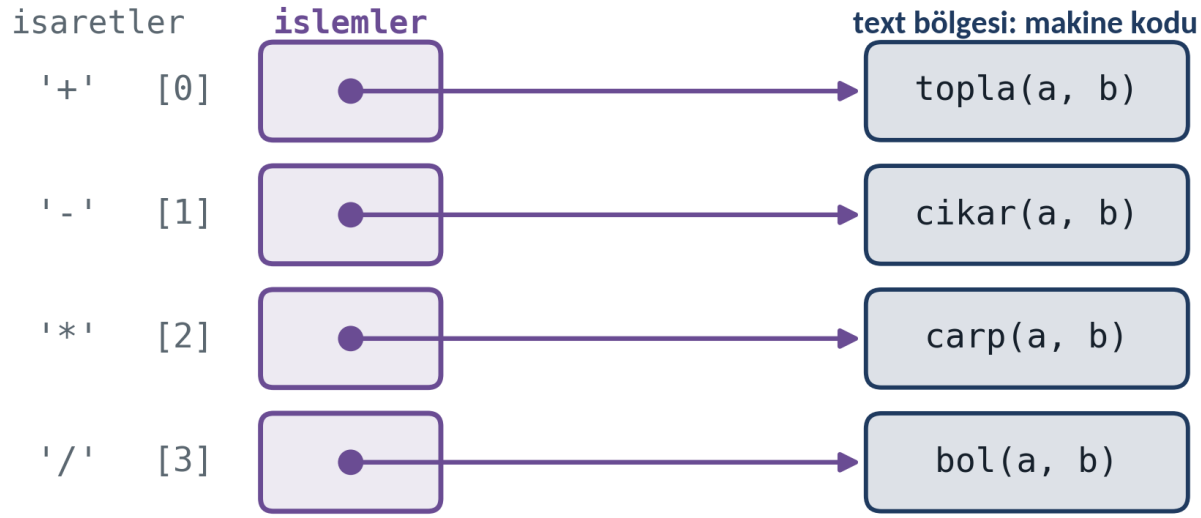
- Her çağrı 1000 `int` ayırır, **free etmez**
- Sızıntı ancak **araçla** görünür
- Kural: her `malloc`'un **tek** `free`'si

GERÇEK İLETİ VE ÇALIŞTIRMA

```
$ ./ha9
sonuç: 1498.5
$ ./ha9_asan   # LeakSanitizer
SUMMARY: AddressSanitizer: 12000 byte(s) leaked in 3 allocation(s).
$ valgrind --leak-check=full ./ha9
12,000 bytes in 3 blocks are definitely lost in loss record 1 of 1
```

Örnek 14: fonksiyon işaretçileri ve işlem tablosu

```
double (*islemler[])(double, double) = {topla, cikar, carp, bol};
```



islemler[i](12, 4) · uygula(islemler[i], x, y): davranış VERİ gibi seçilir

- İçten dışa: “iki `double` alıp `double` döndüren fonksiyonu gösterir”
- Tablo: “12 + 4 = 16.00” … “12 / 4 = 3.00”
- `uygula(islem, a, b)`: davranış **parametre**
- AP-2: `qsort`’un karşılaştırma fonksiyonu

H12_14_fonksiyon_isaretcisi.c · satır 43–45

```
43      double (*islemler[])(double, double) = {topla, cikar, carp, bol};
44      const char isaretler[] = {'+', '-', '*', '/'};
45      int adet = (int)(sizeof islemler / sizeof islemler[0]);
```

Mühendis Gözüyle: sahiplik, araçlar ve bellek güvenliği



Sahiplik

- Her öbek bloğunun **tek sahibi**
- Blok döndüren fonksiyon: bırakma çağırana
- CERT MEM31-C, MEM30-C



Tek temizlik yolu

- Erken `return` = bırakılmamış blok
- Mini-OBS: tek `kayitlari_birak`
- Hata bir bayrakla taşınır



Araçla doğrulama

- `-fsanitize=address,undefined`
- `valgrind --leak-check=full`
- Windows: WSL ya da MSVC `/fsanitize=address`

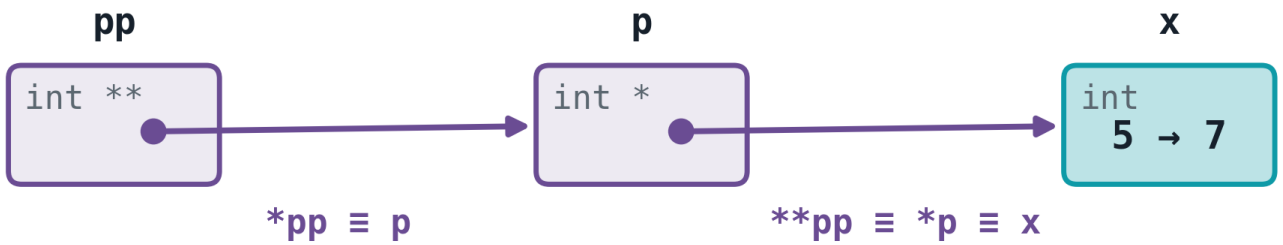


Tanımsız davranış

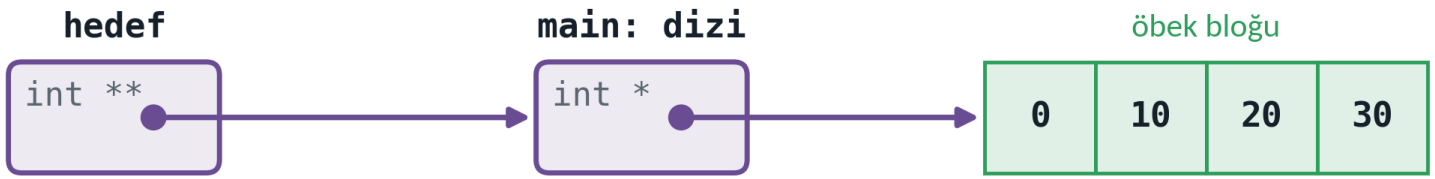
- “Bende çalışıyor” kanıt değil
- HA3 çökmedi, HA8’de GCC NULL döndürdü
- CERT EXP33-C, EXP34-C, DCL30-C

Merak Kutusu: işaretçiye işaretçi

```
int **pp = &p;    **pp = 7;
```



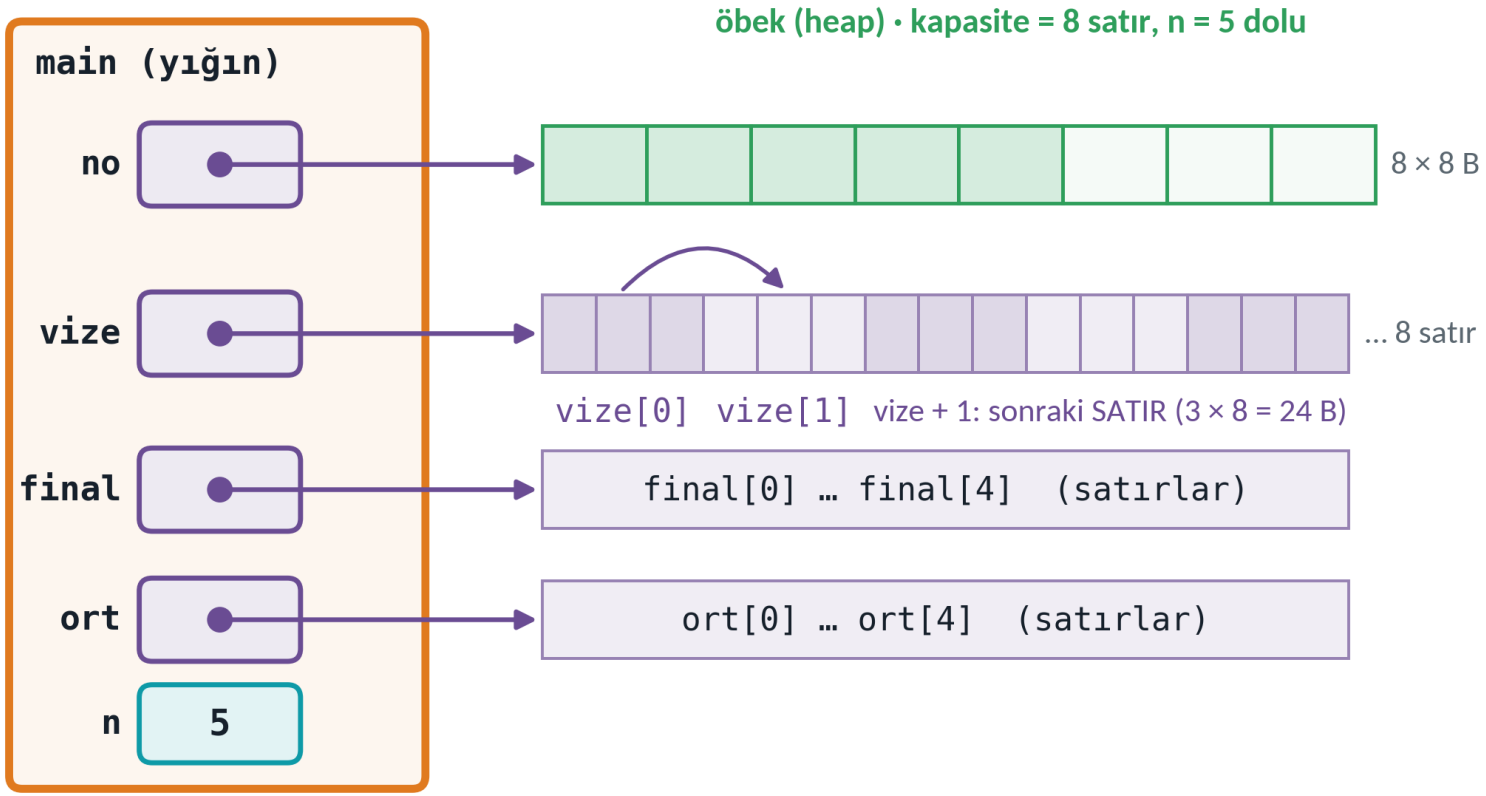

```
dizi_ayir(&dizi, 4):  *hedef = yeni;   → main'in İŞARETÇİSİ değişir
```



- `int **pp = &p;` → `*pp` = p, `**pp` = x
- Çağıranın **işaretçisini** değiştirmek için
- `dizi_ayir(&dizi, 4)` → main'deki `dizi` yeni bloğu gösterir
- Gerçek: "x = 7, *p = 7, **pp = 7", "dizi[3] = 30"
- `char *argv[]`: işaretçi dizisi (H13)

Mini-OBS v0.12: kayıtlar öbekte, kapasite çalışma zamanında

```
double (*vize)[DERS_SAYISI] = malloc(kapasite * sizeof *vize);
```



- Başta **kapasite** sorulur; dolunca `realloc` ile **2 kat**
- `double (*vize)[DERS_SAYISI]`: v0.11 fonksiyonları **değişmeden**
- Büyütme: `g = buyut(…)`, başarıda `vize = g` (**geçici işaretçi**)
- İstatistik: 6 **çıktı parametresi**
- Tek temizlik yolu: `kayitlari_birak`
- `make test`: **79 doğrulama**
- T2 (kapasite 2, 5 öğrenci) Valgrind: 17 allocs, 17 frees, 9,432 bytes
- v0.11 ile fark: yalnızca **sürüm satırı** ve **kapasite sorusu**

`Dolunca (her blok): g = buyut(vize, yeni, sizeof *vize); if (g != NULL) vize = g;`

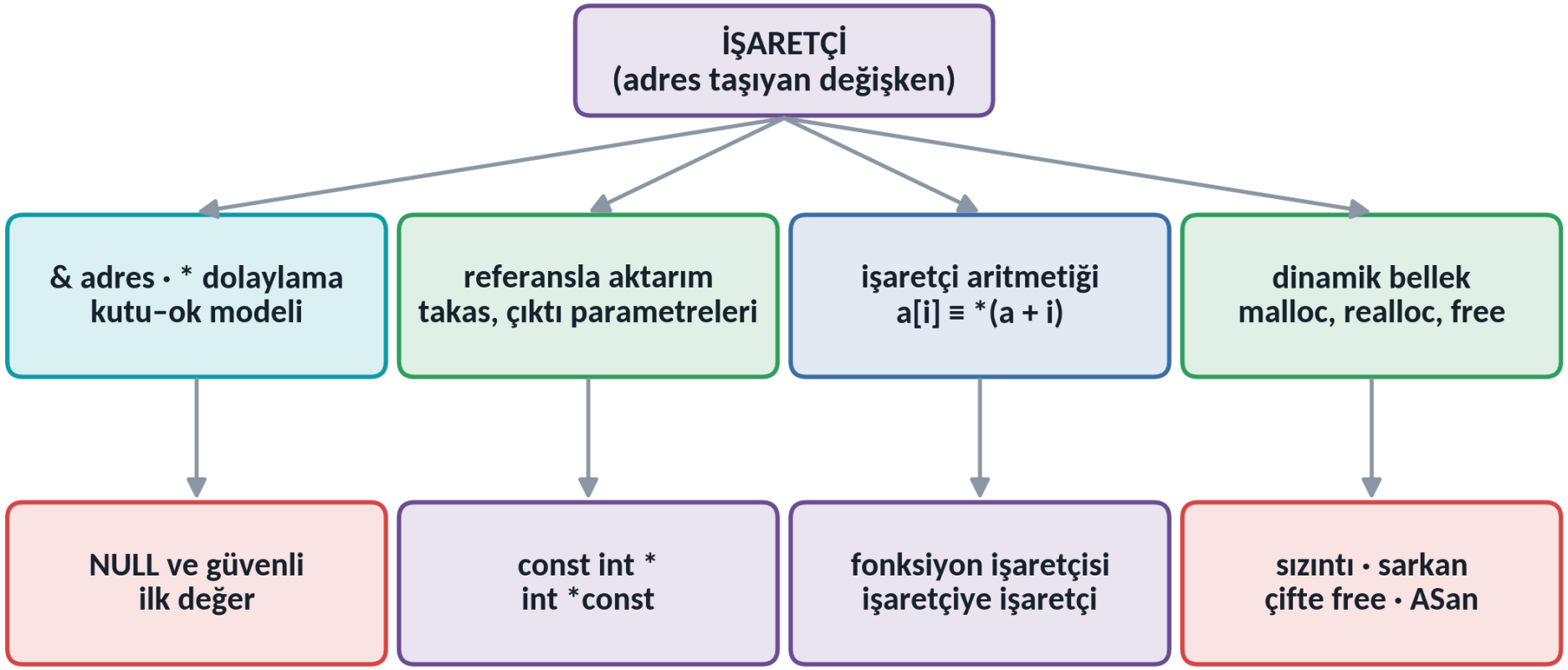
realloc sonucu önce GEÇİCİ işaretçiye; biri başarısız olursa kapasite artırılmaz, eski bloklar geçerli.

AP-2 KÖPRÜSÜ

AP-2 Köprüsü: bu haftanın yapı taşları

- `takas(int *, int *)`: bütün **yerinde sıralama** algoritmalarının temel işlemi
- Fonksiyon işaretçileri → `qsort` ve **karşılaştırma fonksiyonu**
- realloc ile büyüyen dizi → **amortize analiz** (ikiye katlama neden ucuz?)
- İşaretçiler → **bağlı liste**, yığın (stack ADT) ve kuyruk
- İki işaretçi tekniği (Örnek 8) → “two pointers” problemleri

Özet ve kavram haritası



Kaputun altında: bellek = bayt dizisi · 8 baytlık adres · yığın / öbek · sanal bellek, MMU, segfault

AP-2: yerinde sıralama (takas) · qsort (fonksiyon işaretçisi) · bağlı liste, yığın, kuyruk (işaretçiler)

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 Alıştırma 12.2'de `p = a + 1` iken `*(p + 2)` ile `p[-1]` hangi elemanlardır?

2 `dizi = realloc(dizi, yeni)` neden tehlikelidir? Doğrusunu bir cümleyle yazın.

3 Bir fonksiyon bir dizinin hem toplamını hem en büyüğünü döndürmeli: prototipini yazın.

Haftaya hazırlık ve kaynaklar

Okuma

- King (2008), Bölüm 11–12 ve 17
- Kernighan & Ritchie (1988), Bölüm 5
- Ders notu Bölüm 3–5

Uygulama

- Alıştırma 12.7–12.11
- Mini-OBS v0.12: `make asan`, `make valgrind` çıktısıyla teslim
- Web sayfasındaki kutu–ok ve bellek haritası simülasyonları

H13'e köprü

- Karakter dizisi: sonu `'\0'` olan `char` dizisi
- `strlen`, `strcpy` ve taşma
- Mini-OBS v0.13: öğrenci adları

Kodlar: `kodlar/` · gerçek çıktılar: `kodlar/ciktilar/` · tam ASan/Valgrind raporları: `hata_avcisi/ciktilar/`